

Machine Learning

Boosting Techniques: AdaBoost, Gradient Boosting



Dr. G. Omprakash

KLEF Hyderabad (Aziz Nagar) – Dept. of ECE

Machine Learning (25SC2107E)

Session Outcome

- Understand how weak models added one at a time — *AdaBoost*, *gradient boosting*, *XGBoost*, *LightGBM* — become a very strong model.

Boosting

Boosting

Boosting is a machine learning method that builds a strong model by combining many weak models. It works step by step, where each new model tries to fix the mistakes made by the ones before it, so the overall model keeps getting better.

- Models are trained one after another, and each one learns from the errors of the last.
- Data points that are hard to predict correctly get more attention, so the model focuses more on the tricky cases.
- All the weak models are combined, giving more weight to the better performing ones, to create one strong and accurate model.

Key Terms — Boosting

Definitions

Boosting — building an ensemble one model at a time, where each new model is trained to fix what the models before it got wrong.

Weak learner — a deliberately simple model, only a little better than guessing. In boosting these are shallow trees (depth 1 to 4).

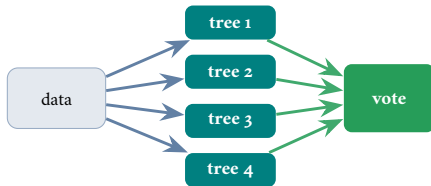
One-split tree (also called a decision stump) — the weakest useful tree, asking a single yes/no question, i.e. depth 1.

Additive model — a model formed by *summing* its parts, $F(x) = h_1(x) + h_2(x) + \dots + h_B(x)$, rather than by voting.

Sequential ensemble — one in which model m cannot be built until model $m - 1$ is finished (the opposite of bagging, which is parallel).

Bagging Works in Parallel, Boosting Works in a Queue

BAGGING — all at once



trees never see each other
(reduces variance)

A bagged model is a *vote*.

$$F(x) = \text{majority}\{h_1(x), h_2(x), \dots, h_B(x)\}$$

BOOSTING — one after another



each tree sees the last one's errors
(reduces bias)

A boosted model is a *sum*:

$$F(x) = h_1(x) + h_2(x) + \dots + h_B(x).$$

Learning From Mistakes

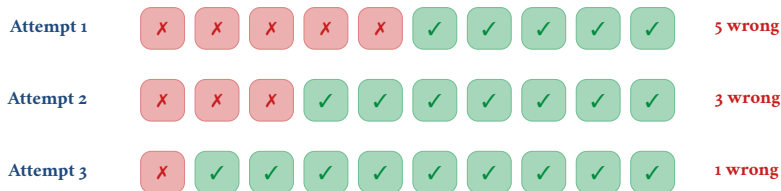


Figure: Each attempt you redo only the questions you got wrong

Types of Boosting Algorithms

- **AdaBoost** (Adaptive Boosting):

AdaBoost gives more weight to the data points that got predicted wrong after each round, so the next model pays more attention to those harder cases. It works especially well for problems where you are choosing between two classes.

- **Gradient Boosting**

Gradient Boosting trains each new model to fix the errors left by the previous group of models, using a method called gradient descent to reduce those errors as much as possible. It can be used for both predicting numbers and predicting categories.

- **XGBoost** (Extreme Gradient Boosting)

XGBoost is a faster and more efficient version of gradient boosting. It can run computations in parallel, avoid overfitting through regularization, and handle large datasets smoothly, which makes it a popular choice for real world machine learning tasks.

Advantages of Boosting

- Reduces bias: It combines simple models one after another, which helps fix the problem of a single simple model being too basic to catch patterns in the data.
- Focuses on hard cases: It pays extra attention to the data points that were predicted wrong, which helps improve the model overall.
- Improves accuracy: It usually predicts better than a single model, and often better than some other ensemble methods too.
- Works for different tasks: It can be used for both predicting categories and predicting numbers.
- Efficient with features: It naturally focuses on the features that actually help with prediction, which can reduce unnecessary complexity and training time.

AdaBoost



AdaBoost (Adaptive Boosting) is a technique that combines several weak models to build one strong model. It works by training models one after another, with each new model paying more attention to the data points that earlier models got wrong.

- Models are trained one after another, each one fixing the mistakes of the last.
- Data points that get predicted wrong are given more weight, so future models focus on them.
- The final prediction is made by combining all the models, giving more say to the better performing ones.

AdaBoost: Working

AdaBoost starts by treating every data point equally, then keeps adjusting these weights after each round, giving more importance to the points that were predicted wrong so the next model focuses on them. This gradually reduces both bias and variance, which is why it works well for classification tasks, though it can struggle when there is a lot of noise or outliers in the data.

- 1 First model: A first simple classifier is trained on the data with every point weighted equally
- 2 Adjusting weights: The points that were misclassified in step 1 are given more weight, so a new classifier is trained that focuses more on them
- 3 Refining: The points still misclassified after step 2 are given even more weight, and another classifier is trained, reducing the misclassified points further to 4.
- 4 Final model: All the earlier classifiers are combined into one final model, which performs better than any single one of them on its own.

Key Terms — AdaBoost

Definitions

Sample weight w_i — a number attached to each row, saying how much that row matters when the next model is fitted. All rows start equal at $1/n$.

Weighted error ϵ_m — the total weight of the rows that model m got wrong, rather than a plain count of them.

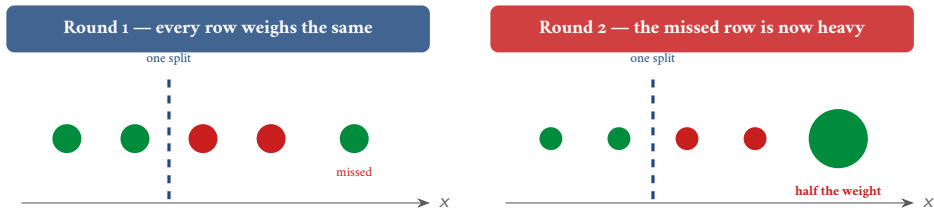
Model weight α_m — how loudly model m votes in the final answer, $\alpha_m = \frac{1}{2} \ln \frac{1-\epsilon_m}{\epsilon_m}$. A coin-toss model gets $\alpha_m = 0$.

Reweighting — after each round, *increasing* the weights of misclassified rows and *decreasing* those of correct ones, so the next model concentrates on the hard cases.

Weighted vote — the final prediction $H(x) = \text{sign}(\sum_m \alpha_m h_m(x))$.

AdaBoost

Let the training dataset contain N samples, denoted as $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$, where x_i represents the features and $y_i \in \{-1, 1\}$ represents the binary class label



$$\varepsilon_m = \sum_{i: h_m(x_i) \neq y_i} w_i \quad \alpha_m = \frac{1}{2} \ln \frac{1 - \varepsilon_m}{\varepsilon_m} \quad \tilde{w}_i = w_i e^{-\alpha_m y_i h_m(x_i)}$$

$$\text{renormalise} \quad Z_m = \sum_{i=1}^n \tilde{w}_i \quad w_i \leftarrow \frac{\tilde{w}_i}{Z_m} \quad \text{so that} \quad \sum_{i=1}^n w_i = 1$$

- Final answer: $H(x) = \text{sign}(\sum_m \alpha_m h_m(x))$.

Example: One Round of AdaBoost

Example. Five points, all weights start at 0.2. The one-split tree asks $x \leq 2.5$.

$x = 1$	$x = 2$	$x = 3$	$x = 4$	$x = 5$
$y = +1$ $h_1 = +1$ ✓ correct	$y = +1$ $h_1 = +1$ ✓ correct	$y = -1$ $h_1 = -1$ ✓ correct	$y = -1$ $h_1 = -1$ ✓ correct	$y = +1$ $h_1 = -1$ ✗ wrong
$w : 0.2 \rightarrow 0.125$	$w : 0.2 \rightarrow 0.125$	$w : 0.2 \rightarrow 0.125$	$w : 0.2 \rightarrow 0.125$	$w : 0.2 \rightarrow \mathbf{0.500}$

- Weighted error $\varepsilon_1 = 0.2$, so Model weight $\alpha_1 = \frac{1}{2} \ln \frac{0.8}{0.2} = \frac{1}{2} \ln 4 = 0.693$.
- Weight update: the four correct rows get $\tilde{w} = 0.2e^{-0.693} = 0.10$; the wrong row gets $\tilde{w} = 0.2e^{+0.693} = 0.40$.
- Renormalised weights: $Z_1 = \sum_i \tilde{w}_i = 4(0.10) + 0.40 = 0.80$, then divide every row by Z_1 :

$$\text{correct } \frac{0.10}{0.80} = 0.125 \quad \text{wrong } \frac{0.40}{0.80} = 0.500 \quad \text{check } 4(0.125) + 0.500 = 1.000 \checkmark$$

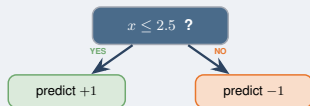
The single missed point now carries half of all the weight.



(DEEMED TO BE UNIVERSITY)

INSIDE ONE ROUND — WHAT THE TREE DOES TO EACH ROW

round 1 of AdaBoost, row by row



the whole tree of round 1 — one question

The tree is the **same** for every row. It knows one question, $x \leq 2.5$.

Each row answers that question *with its own* x , falls into one leaf, and takes that leaf's label as its prediction. We then compare the prediction with the truth y .

Every row starts with the same weight $w = 0.2$ — in round 1 no row is special yet.

the row	answers the question	lands in	tree says	truth	verdict	its weight is rescaled	after $\div Z_1$
$x = 1, y = +1$	$1 \leq 2.5$ yes	left leaf	+1	+1	correct	$0.2 e^{-0.693} = 0.10$	0.125 (down)
$x = 2, y = +1$	$2 \leq 2.5$ yes	left leaf	+1	+1	correct	$0.2 e^{-0.693} = 0.10$	0.125 (down)
$x = 3, y = -1$	$3 \leq 2.5$ no	right leaf	-1	-1	correct	$0.2 e^{-0.693} = 0.10$	0.125 (down)
$x = 4, y = -1$	$4 \leq 2.5$ no	right leaf	-1	-1	correct	$0.2 e^{-0.693} = 0.10$	0.125 (down)
$x = 5, y = +1$	$5 \leq 2.5$ no	right leaf	-1	+1	WRONG	$0.2 e^{+0.693} = 0.40$	0.500 (up)

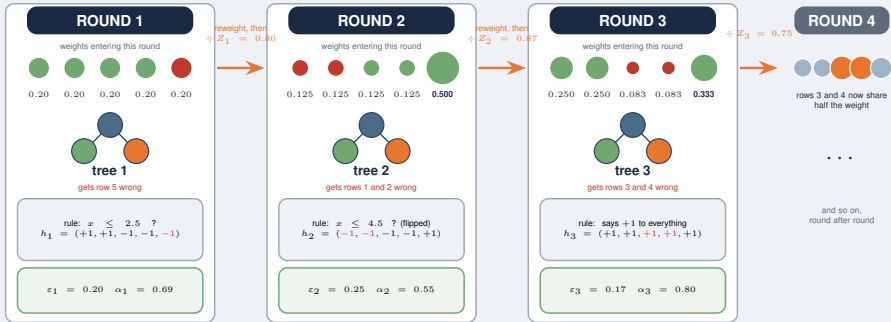
Only the **red row** matters for what comes next: its weight is the whole of $\varepsilon_1 = 0.2$, which fixes the tree's vote $\alpha_1 = \frac{1}{2} \ln \frac{0.8}{0.2} = 0.693$ and therefore the size of every rescaling above.

$Z_1 = 4(0.10) + 0.40 = 0.80$ puts the column back to a total of 1 — and the one row the tree missed now holds **half** of it.

ADABOOST — HOW THE WEIGHTS TRAVEL FROM TREE TO TREE

same five rows every round; only their weights change

each circle is one training row — its area is that row's weight green = this tree got it right red = this tree got it wrong



$$\text{Final model: } H(x) = \text{sign}(\alpha_1 h_1(x) + \alpha_2 h_2(x) + \alpha_3 h_3(x) + \dots)$$

Gradient Boosting

Gradient Boosting builds models one after another, where each new model tries to fix the errors of the previous ones using a method called gradient descent. It keeps checking how wrong the predictions are, adds a new model to correct that, and repeats this until the result is good enough.

- Shrinkage controls how much each new model affects the final result, using a setting called the learning rate.
- A small learning rate means slower but safer learning, needing more models.
- A large learning rate means faster learning, but with a higher risk of overfitting.

Key Terms — Gradient Boosting

Definitions

Residual — what is left over, $r_i = y_i - F(x_i)$: the part of the answer the model so far has not explained.

Loss function L — the quantity being minimised, e.g. squared error $\frac{1}{2}(y - \hat{y})^2$ for regression, log-loss for classification.

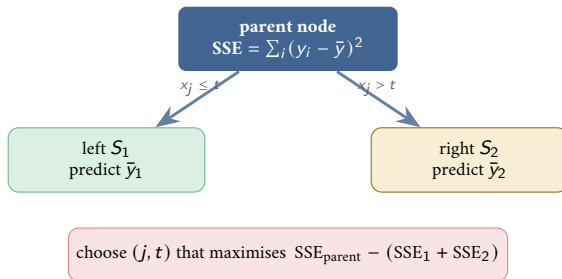
Functional gradient descent — treating the predictions as the thing being optimised: each new tree is fitted to the negative gradient $-\partial L / \partial F$, which for squared error is the residual.

Learning rate η (shrinkage) — the fraction of each new tree that is actually kept, $F \leftarrow F + \eta h$. Small η means many small, safe steps.

Early stopping — halting when a validation score stops improving, instead of fixing the number of trees by hand.

Subsampling (stochastic boosting) — fitting each tree on a random fraction of the rows, which adds bagging-style randomness.

Recap: Regression



- Try every feature j and every candidate threshold t ; keep the pair with the largest drop in SSE.
- Weighted MSE after the split: $\frac{n_1}{n} \text{MSE}_1 + \frac{n_2}{n} \text{MSE}_2$.

Quick Check — Regression Trees

Questions

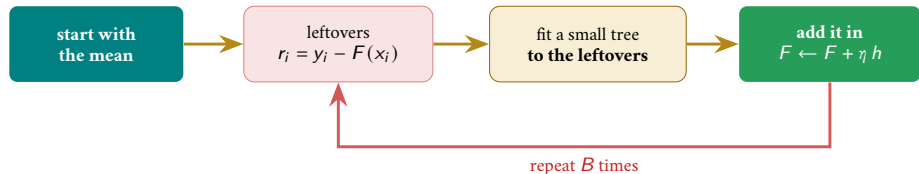
- 1 A leaf holds $y = \{4, 8, 12\}$. Give its prediction and its SSE.

Ans. Prediction = $\bar{y} = 8$. SSE = $(4 - 8)^2 + (8 - 8)^2 + (12 - 8)^2 = 16 + 0 + 16 = 32$.

- 2 Split $y = \{4, 8, 12\}$ into $S_1 = \{4, 8\}$, $S_2 = \{12\}$. What is the SSE reduction?

Ans. $SSE_1 = (4 - 6)^2 + (8 - 6)^2 = 8$, $SSE_2 = 0$. Reduction = $32 - 8 = 24$.

Gradient Boosting



- **Why “gradient”?** For $L = \frac{1}{2}(y - F)^2$ we have $-\partial L / \partial F = y - F$ — the residual is the gradient step.
- AdaBoost changes the *weights* of the rows; gradient boosting changes the *target* of the next tree.

INSIDE ONE ROUND OF GRADIENT BOOSTING

round 1, row by row — and where the leaf values come from

Gradient boosting changes the target: the new tree is fitted to the leftovers
 $r_i = y_i - F_0(x_i)$.

The tree splits at $x \leq 4.5$.

$$\begin{array}{l} \text{left leaf } (x \leq 4.5: \text{rows 1-4}) \\ -13.5 - 11.5 - 3.5 + 1.5 \\ \hline 4 \\ -6.75 \end{array} =$$

$$\begin{array}{l} \text{right leaf } (x > 4.5: \text{rows 5-6}) \\ 10.5 + 16.5 \\ \hline 2 \\ +13.5 \end{array} =$$

the row	model so far	leftover	$x \leq 4.5$?	leaf value	updated model	new leftover
$x = 1, y = 10$	$F_0 = 23.5$	-13.5	yes, left	-6.75	$23.5 - 6.75 = 16.75$	-6.75
$x = 2, y = 12$	$F_0 = 23.5$	-11.5	yes, left	-6.75	$23.5 - 6.75 = 16.75$	-4.75
$x = 3, y = 20$	$F_0 = 23.5$	-3.5	yes, left	-6.75	$23.5 - 6.75 = 16.75$	+3.25
$x = 4, y = 25$	$F_0 = 23.5$	+1.5	yes, left	-6.75	$23.5 - 6.75 = 16.75$	+8.25
$x = 5, y = 34$	$F_0 = 23.5$	+10.5	no, right	+13.5	$23.5 + 13.5 = 37.0$	-3.0
$x = 6, y = 40$	$F_0 = 23.5$	+16.5	no, right	+13.5	$23.5 + 13.5 = 37.0$	+3.0

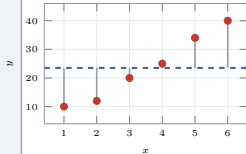
GRADIENT BOOSTING — THE MODEL AFTER EACH ROUND

six rows; each new tree is fitted to what is left over

the red dots are the data y ; the coloured line is the current model F_m ; each grey stick is one residual $r_i = y_i - F_m(x_i)$

STAGE 0 — F_0

just the mean of y

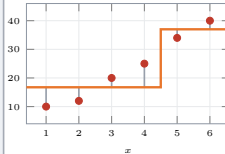


MSE = 118.6

$+ h_1(x)$

STAGE 1 — F_1

F_0 plus tree 1

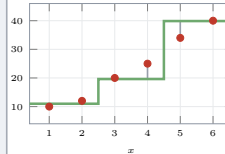


MSE = 27.5

$+ h_2(x)$

STAGE 2 — F_2

F_1 plus tree 2



MSE = 10.9

$F_0(x) = \bar{y} = 23.5$. After that $F_m(x) = F_{m-1}(x) + \eta h_m(x)$, where h_m is a one-split tree fitted to the residuals $r_i = y_i - F_{m-1}(x_i)$. Here $\eta = 1$, so the whole tree is added; in practice $\eta \approx 0.1$ and you need many more rounds.

tree 1: $x \leq 4.5 \Rightarrow -6.75$, else $+13.5$

tree 2: $x \leq 2.5 \Rightarrow -5.75$, else $+2.875$

Quick Check — The Boosting Update

Direct questions

- 1 In gradient boosting with squared-error loss, what exactly is the target of tree m ?

Ans. The residual $r_i = y_i - F_{m-1}(x_i)$ — the part of y not yet explained, which for this loss equals the negative gradient $-\partial L / \partial F$.

- 2 Write the update rule and say what η does to it.

Ans. $F_m(x) = F_{m-1}(x) + \eta h_m(x)$. η keeps only a fraction of each tree, so the model takes many small steps instead of a few large ones.

- 3 What is $F_0(x)$ for a regression problem?

Ans. The constant that minimises the loss over all rows — for squared error, the mean $\bar{y}$.

Household Electricity Bills

Case Study

A power utility estimates each household's monthly electricity bill $y \in \mathbb{R}$ (in thousands of rupees) from smart-meter readings, appliance counts, floor area and weather. The team builds a single regression tree baseline and a gradient boosted model.

- (a) Explain how the regression tree selects its split points using MSE, and how the predicted value at a terminal leaf is computed.
- (b) Take three households with bills $y = \{2.0, 6.0, 4.0\}$ and start from the constant $F_0(x) = \bar{y}$. Compute the residual vector $r_{i,1}$. Then, if the first weak learner outputs $\gamma = (-1.5, +1.5, 0)$ and the learning rate is $\eta = 0.1$, compute $F_1(x)$ and the new residuals.

Solution (a) and (b)

(a) Splits and leaf values

Choosing a split: The tree tries every feature and every possible cut point on it, such as floor area above or below some value. Each cut divides the households into two groups, and the tree measures the mean squared error inside each group, which shows how far the actual bills are from that group's average bill. The split with the smallest total error across both groups is chosen, since that is the cut making the two groups most similar internally.

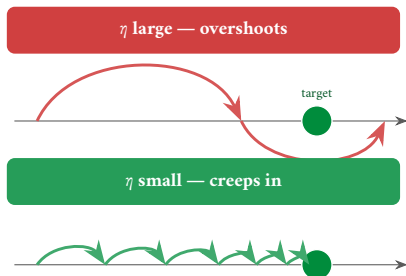
Prediction at a leaf: Each terminal leaf holds a few households, and the prediction is simply the average of their actual bills. For example, a leaf with bills of 4.0, 4.4 and 4.8 predicts 4.4 thousand rupees. The average is used because it is the value that minimises the mean squared error within that leaf.

(b) One boosting step

$$F_0(x) = \bar{y} = \frac{2.0 + 6.0 + 4.0}{3} = 4.0$$

	house 1	house 2	house 3
y	2.0	6.0	4.0
F_0	4.0	4.0	4.0
$r_{i,1} = y_i - F_0$	-2.0	+2.0	0.0
γ (tree 1)	-1.5	+1.5	0
$F_1 = F_0 + \eta\gamma$	3.85	4.15	4.00
$r_{i,2} = y_i - F_1$	-1.85	+1.85	0.00

The Learning Rate η



η	Train	Test
0.01	0.871	0.808
0.05	0.971	0.817
0.10	0.998	0.812
0.30	1.000	0.783
1.00	1.000	0.808

200 trees in every row

- η and B pull against each other: *halve η , roughly double the trees*. Typical η : 0.01–0.1.
- Small η with many trees is the safe combination

AdaBoost vs Gradient Boosting

AdaBoost

- Corrects misclassified data points
- Gives more weight to wrong points
- Uses weight adjustment
- Combines models by weighted voting
- Sensitive to noisy data
- Mainly used for classification

Gradient Boosting

- Corrects residual errors
- Trains a new model to predict residuals
- Uses gradient descent
- Combines models by summing scaled predictions
- Less sensitive, but outliers still affect residuals
- Used for both regression and classification

Gradient Boosting in Code

```
1 from sklearn.ensemble import \
2     GradientBoostingClassifier
3
4 gb = GradientBoostingClassifier(
5     n_estimators=1000,
6     learning_rate=0.05,    # eta
7     max_depth=3,          # keep it WEAK
8     subsample=0.8,
9     validation_fraction=0.15,
10    n_iter_no_change=20,   # early stop
11    random_state=0)
12
13 gb.fit(Xtr, ytr)
14
15 print(gb.n_estimators_)    # trees used
16 print(gb.score(Xte, yte))
```

- `learning_rate` — start at 0.05.
- `max_depth` 2 to 4: the trees must stay weak.
- `subsample=0.8` — adds bagging-style randomness.
- `n_iter_no_change` — stops when validation stops improving.

For regression, use
`GradientBoostingRegressor` with the same arguments.